Agentic架构6: Planner → Executor → Verifier (PEV)

> **规划器 → 执行器 → 校验器（PEV）** 架构 —— 一种为智能体系统引入关键**鲁棒性与自我修正能力**的模式。该架构的灵感来源于**严谨的软件工程与质量保证流程**，即一项工作只有在通过**验证**后才能被视为“完成”。
>
> 尽管标准的规划型智能体能够带来结构性和可预测性，但它基于一个关键假设：**其工具将完美运行，并且每次都能返回有效数据**。然而在现实世界中，API可能失效、搜索可能返回空结果、数据格式也可能出现错误。PEV模式通过引入一个**专门的校验器智能体**来应对这一问题，该智能体在每次执行后充当质量检查环节，使系统能够**检测到故障并动态恢复**。

定义

规划器 → 执行器 → 校验器（PEV） 架构是一种三阶段工作流，它将**规划**、**执行和校验**三个环节明确分离。该架构确保每个步骤的输出在智能体继续前进之前都经过**验证**，从而形成一个**鲁棒的**、**自我修正的循环**。

高级工作流程

1. **规划：** “**规划器**”智能体将高层次目标分解为**一系列具体**、**可执行的步骤序列**。
2. **执行：** “**执行器**”智能体从规划中取出**下一个步骤**，并**调用相应的工具**。
3. **校验：** “**校验器**”智能体检查执行器的输出结果，**验证其正确性**、**相关性以及是否存在潜在错误**，并给出判断：该步骤是成功还是失败？
4. **路由与迭代：** 根据校验器的判断，**路由器**决定下一步操作：
    - 如果步骤**成功**且规划尚未完成，则返回执行器执行下一步。
    - 如果步骤**失败**，则返回规划器生成**新的规划**，通常会附带**失败上下文信息**，以便新规划更加智能。
    - 如果步骤**成功**且规划已完成，则进入最终的整合步骤。

适用场景 / 应用

- **安全关键型应用（金融、医疗）：** 当错误代价高昂时，PEV 提供了必要的**防护机制**，防止智能体基于错误数据采取行动。
- **工具不可靠的系统：** 当处理可能存在**不稳定**或返回数据**不一致**的外部 API 时，校验器可以优雅地**捕获失败**。
- **高精度任务（法律、科学）：** 对于需要高度事实准确性的任务，校验器确保每条检索到的信息**在用于后续推理之前都是有效的**。

优势与劣势

- **优势：**
    - **鲁棒性与可靠性：** 其核心优势在于能够**检测并从错误中恢复**。
    - **模块化：** 关注点分离使得系统更易于**调试和维护**。
- **劣势：**
    - **延迟与成本增加：** 每次执行后都增加一个校验步骤，意味着**更多的大语言模型调用次数**，使其成为我们目前介绍过的架构中速度最慢、成本最高的一种。

- **校验器复杂性：** 设计一个有效的校验器可能具有挑战性。它需要足够智能，能够**区分微小问题与关键性失败**。

阶段 0：基础与环境搭建

首先安装所需的库，并为 Nebius、LangSmith 以及相关工具配置 API 密钥。

步骤 0.1：安装核心库

安装本系列项目所需的**标准库套件**

```
pip install -q -U langchain-nebius langchain langgraph rich python-dotenv
langchain-tavily
```

步骤 0.2：导入库并配置密钥

导入必要的模块，并从 `.env` 文件中加载 API 密钥。

需要操作： 请在此目录下创建一个 `.env` 文件，并填入密钥：

```
NEBIUS_API_KEY="你的_nebius_api_密钥"
LANGCHAIN_API_KEY="你的_langsmith_api_密钥"
TAVILY_API_KEY="你的_tavily_api_密钥"
```

```
"""
PEV架构 - 计划-执行-验证 (Plan-Execute-Verify) 智能体系统
该模块实现了基于计划-执行-验证范式的智能体工作流，提供更可控和可验证的任务执行机制
"""

import os
import re
from typing import List, Annotated, TypedDict, Optional
from dotenv import load_dotenv
import json

# ==================== LangChain 组件导入 ====================
# LangChain提供LLM抽象、工具集成和数据结构等核心功能
from langchain_nebius import ChatNebius          # Nebius云平台的大语言模型接
口
from langchain_tavily import TavilySearch        # Tavily搜索引擎工具，用于获
取实时网络信息
from langchain_core.messages import BaseMessage, ToolMessage  # 消息类型定
义，ToolMessage用于工具执行结果
from pydantic import BaseModel, Field            # Pydantic用于数据验证和结构
化输出定义

# ==================== LangGraph 组件导入 ====================
```

```
# LangGraph提供基于图的状态机框架，用于构建复杂的智能体工作流
from langgraph.graph import StateGraph, END      # 状态图核心组件和终止节点

# ==================== 可视化与调试工具 ====================
# Rich库提供增强的终端输出格式，便于调试和演示
from rich.console import Console                 # 富文本控制台输出
from rich.markdown import Markdown               # Markdown格式渲染

# ==================== API密钥与环境配置 ====================
"""
PEV架构环境变量配置说明：
- NEBIUS_API_KEY: Nebius LLM服务的认证密钥
- LANGCHAIN_API_KEY: LangChain追踪服务的认证密钥
- TAVILY_API_KEY: Tavily搜索API的访问密钥
- LANGCHAIN_PROJECT: LangChain追踪项目标识符（设置为PEV专用项目）
"""

load_dotenv()  # 从.env文件加载环境变量

# 配置LangChain追踪系统，用于监控和调试PEV智能体行为
os.environ["LANGCHAIN_TRACING_V2"] = "true"       # 启用V2版本的追踪功能
os.environ["LANGCHAIN_PROJECT"] = "Agentic Architecture - PEV (Nebius)"  #
设置PEV专用追踪项目

# 验证关键环境变量是否存在，缺失时给出提示，这三个API密钥是系统正常运行的必要条件
for key in ["NEBIUS_API_KEY", "LANGCHAIN_API_KEY", "TAVILY_API_KEY"]:
    if not os.environ.get(key):
        print(f"{key} not found. Please create a .env file and set it.")

print("Environment variables loaded and tracing is set up.")
```

阶段 1：基线对比——规划器-执行器智能体

要理解校验器的必要性，首先需要构建一个**没有校验器的智能体**。该智能体会**创建计划并盲目执行**，从而展示**当工具调用出错时可能发生的失败情况**。

步骤 1.1：构建规划器-执行器智能体

构建一个简单的规划器-执行器工作流图。为了模拟现实世界中的故障，创建一个特殊的 **“不稳定”工具**。该工具会针对特定查询故意返回错误信息，而我们的基础智能体将无法处理这种情况。

```
# ==================== 控制台与LLM初始化 ====================
# 初始化Rich控制台实例，用于增强终端输出格式和可视化效果
console = Console()

# 初始化Nebius LLM实例，配置模型参数
llm = ChatNebius(model="meta-llama/Meta-Llama-3.1-8B-Instruct",
temperature=0)

# ==================== 模拟不稳定工具定义 ====================
```

```
# 定义一个会针对特定查询失败的工具，用于测试PEV架构的鲁棒性
# 该工具模拟了真实世界中API可能出现的故障场景
def flaky_web_search(query: str) -> str:
    """
    模拟不稳定网络搜索工具
    这种设计模拟了生产环境中可能遇到的不稳定外部依赖，用于验证智能体在遇到工具故障时的处理能力。

    参数:query: 搜索查询字符串
    返回:str: 搜索结果或错误信息的字符串表示
    """
    console.print(f"--- TOOL: Searching for '{query}'... ---")

    # 模拟特定查询的API故障
    if "employee count" in query.lower():
        console.print("--- TOOL: [bold red]Simulating API failure![/bold red] ---")
        return "Error: Could not retrieve data. The API endpoint is currently unavailable."
    else:
        # 正常执行搜索，最多返回2条结果
        result = TavilySearch(max_results=2).invoke(query)
        # 确保返回值始终为字符串类型，便于后续处理
        # 处理可能返回的字典或列表类型，统一转换为JSON字符串
        if isinstance(result, (dict, list)):
            return json.dumps(result, indent=2)
        return str(result)

# ==================== 基础计划-执行智能体状态定义 ====================
# 定义PE架构的状态数据结构，跟踪任务的执行进度
class BasicPEState(TypedDict):
    """
    基础计划-执行智能体状态数据结构，该状态设计遵循"计划-执行-综合"三阶段模式。
    """
    user_request: str                # 原始用户请求，作为所有阶段的输入基准
    plan: Optional[List[str]]        # 待执行的计划步骤列表，每执行一步就移除一项
    intermediate_steps: List[str]    # 中间步骤的执行结果，按执行顺序累积
    final_answer: Optional[str]      # 最终合成的答案，仅在综合阶段后存在

# ==================== 计划输出模型定义 ====================
# 定义计划生成的结构化输出格式，确保LLM输出可解析的计划步骤
class Plan(BaseModel):
    """
    计划步骤结构化模型，该模型强制LLM输出格式化的计划列表
    """
    steps: List[str] = Field(description="A list of tool calls to execute.")

# ==================== 计划节点定义 ====================
def basic_planner_node(state: BasicPEState):
    """
    计划节点 - 将用户请求分解为可执行的步骤列表
    该节点负责任务分解，将复杂的用户请求拆解为一系列原子化的工具调用步骤。这是PE架构的核心规划阶段。
```

```
    参数:state: 当前智能体状态，包含用户请求
    返回:dict: 包含计划步骤列表的状态更新
    """
    console.print("--- (Basic) PLANNER: Creating plan... ---")

    # 配置LLM为结构化输出模式，强制返回Plan格式
    planner_llm = llm.with_structured_output(Plan)

    # 构建计划生成提示词
    # 明确约束：只返回JSON，使用flaky_web_search工具
    prompt = f"""
    You are a planning agent.
    Your job is to decompose the user's request into a list of clear tool
queries.

    - Only return JSON that matches this schema: {{ "steps": [ "query1",
"query2", ... ] }}
    - Do NOT return any prose or explanation.
    - Always use the 'flaky_web_search' tool for queries.

    User's request: "{state['user_request']}"
    """

    # 调用结构化LLM生成计划
    plan = planner_llm.invoke(prompt)

    # 返回计划步骤列表，状态中的plan字段将被更新
    return {"plan": plan.steps}

# ==================== 执行节点定义 ====================
def basic_executor_node(state: BasicPEState):
    """
    执行节点 - 执行计划中的下一个步骤
    该节点负责逐步执行计划中的工具调用，这种设计实现了"逐步消耗"模式，确保计划的顺序执
行。

    参数:state: 当前智能体状态，包含待执行计划和历史步骤
    返回:dict: 更新后的计划（移除第一项）和累积的执行结果
    """
    console.print("--- (Basic) EXECUTOR: Running next step... ---")

    # 取出计划列表的第一个步骤
    next_step = state["plan"][0]

    # 调用模拟不稳定工具执行搜索
    result = flaky_web_search(next_step)

    # 返回状态更新：
    # - plan: 移除已执行的第一步，保留剩余步骤
    # - intermediate_steps: 追加本次执行结果
    return {
        "plan": state["plan"][1:],
        "intermediate_steps": state["intermediate_steps"] + [result]
```

```
    }

# ==================== 综合节点定义 ====================
def basic_synthesizer_node(state: BasicPEState):
    """
    综合节点 - 根据所有执行结果生成最终答案
    该节点负责将执行阶段收集的所有结果整合为最终回答,这是PE架构的输出阶段，将分散的工具结果转化为用户可理解的最终输出。

    参数:state: 当前智能体状态，包含所有中间执行结果
    返回:dict: 包含最终答案的状态更新
    """
    console.print("--- (Basic) SYNTHESIZER: Generating final answer... ---")

    # 将所有中间步骤结果拼接为统一上下文，使用换行符分隔，便于LLM理解
    context = "\n".join(state["intermediate_steps"])

    # 构建综合提示词，要求基于上下文生成答案
    prompt = f"Synthesize an answer for '{state['user_request']}' using this data:\n{context}"

    # 调用LLM生成最终答案
    answer = llm.invoke(prompt).content

    return {"final_answer": answer}

# ==================== PE图结构构建 ====================
# 构建计划-执行-综合三阶段计算图
pe_graph_builder = StateGraph(BasicPEState)

# 添加三个核心节点
pe_graph_builder.add_node("plan", basic_planner_node)           # 计划节点：任务分解
pe_graph_builder.add_node("execute", basic_executor_node)       # 执行节点：工具调用
pe_graph_builder.add_node("synthesize", basic_synthesizer_node) # 综合节点：答案生成

# 设置入口点：从计划节点开始
pe_graph_builder.set_entry_point("plan")

# 添加条件边：计划节点后，根据是否有计划步骤决定下一步
# 如果有计划步骤 -> 执行节点，否则 -> 综合节点
pe_graph_builder.add_conditional_edges(
    "plan",
    lambda s: "execute" if s["plan"] else "synthesize"
)

# 添加条件边：执行节点后，根据是否有剩余计划步骤决定下一步
# 如果还有计划步骤 -> 继续执行；否则 -> 综合节点
pe_graph_builder.add_conditional_edges(
    "execute",
    lambda s: "execute" if s["plan"] else "synthesize"
```

```
)

# 添加固定边：综合节点执行完成后终止
pe_graph_builder.add_edge("synthesize", END)

# ==================== 图编译与应用初始化 ====================
# 将构建的图结构编译为可执行的应用实例
basic_pe_app = pe_graph_builder.compile()

# 输出编译成功状态
print("Basic Planner-Executor agent compiled successfully.")
```

步骤 1.2：在“不稳定”问题上测试基础智能体

给基础智能体分配一个任务，该任务要求其使用我们**已知会失败的特定查询**来调用 `flaky_web_search` 工具。这将展示其无法处理错误的情况。

```
# ==================== 不稳定测试用例配置 ====================
# 构建包含不稳定因素的测试查询，用于评估PE架构的鲁棒性
# 这种测试设计用于验证：智能体如何处理部分工具调用失败、故障传播机制、错误恢复能力
flaky_query = "What was Apple's R&D spend in their last fiscal year, and
what was their total employee count? Calculate the R&D spend per employee."

# ==================== 测试执行与可视化 ====================
# 输出测试开始标识，使用黄色区分不同类型的测试
console.print(
    f"[bold yellow]Testing BASIC P-E agent on a flaky query:[/bold
yellow]\n'{flaky_query}'\n"
)

# ==================== 初始化PE状态 ====================
# 构建计划-执行智能体的初始状态
# 关键设计：
# - user_request: 包含复杂多步骤的用户请求
# - intermediate_steps: 初始为空列表，用于累积执行结果
# - plan: 不需要手动设置，将由计划节点自动生成
# - final_answer: 初始为空，将由综合节点生成
initial_pe_input = {"user_request": flaky_query, "intermediate_steps": []}

# ==================== PE系统调用 ====================
# 执行计划-执行-综合图应用，处理包含不稳定因素的复杂查询
#
# 阶段1 - 计划（PLANNER）：
#   └── 将用户请求分解为步骤列表
# 阶段2 - 循环执行（EXECUTOR）：
#   ├── 执行步骤1: 查询研发支出 → 成功，返回数据
#   └── 执行步骤2: 查询员工总数 → 触发flaky_web_search的故障模拟，返回错误信息
# 阶段3 - 综合（SYNTHESIZER）：
#   └── 基于执行结果生成最终答案（由于步骤2失败，综合节点需要处理不完整的数据）
final_pe_output = basic_pe_app.invoke(initial_pe_input)
```

```
# ==================== 结果输出与渲染 ====================
# 输出最终答案的分隔标识
console.print("\n--- [bold red]Final Output from Basic P-E Agent[/bold red]
---")

# 使用Markdown渲染PE智能体生成的最终答案
# 期望的输出行为：
# - 如果两个工具都成功：显示完整的研发支出/员工计算
# - 如果员工查询失败：可能显示错误信息或不完整的计算结果
# 这展示了PE架构在工具不稳定情况下的输出质量
console.print(Markdown(final_pe_output['final_answer']))
```

> **输出结果讨论：** 如预期般失败了。执行轨迹显示，智能体创建了一个计划，很可能类似于 `["苹果公司上一财年研发支出", "苹果公司员工总数"]`。它成功执行了第一步。然而，在第二步中，我们的 `flaky_web_search` 工具返回了一条错误信息字符串。
>
> 关键失败发生在最后一步。**合成器无法得知第二步已经失败，它将错误信息当作有效数据处理**。因此，其最终答案**毫无意义**，很可能会输出类似“由于其中一个输入是错误信息，我无法执行计算”这样的内容。智能体盲目地执行完整个计划，最终产生了**无用的输出**。这充分说明了**校验步骤**的关键必要性。

阶段 2：进阶方案——规划器-执行器-校验器智能体

构建完整的 PEV 智能体。我们将添加一个专门的**校验器**节点，并创建**更复杂的路由逻辑**，使智能体能够从工具故障中恢复。

步骤 2.1：定义校验器与 PEV 工作流图

1. 定义一个 `VerificationResult` 的 **Pydantic 模型**，用于**规范校验器的结构化输出**。
2. 创建**校验器节点**，该节点将负责**分析执行器的输出结果**。
3. 创建一个新的、更复杂的**路由器**，使其能够处理**校验器的反馈**并触发重新规划的循环。

```
# ==================== 验证结果模型定义 ====================
# 定义验证器的结构化输出格式，用于判断工具执行是否成功
class VerificationResult(BaseModel):
    """
    验证结果结构化模型——用于PEV架构中的验证阶段：
    - is_successful: 布尔值表示工具执行是否成功
    - reasoning: 提供判断的详细理由，增强可解释性
    """
    is_successful: bool = Field(description="True if the tool execution was
successful and the data is valid.")
    reasoning: str = Field(description="Reasoning for the verification
decision.")

# ==================== PEV架构状态定义 ====================
# 定义计划-执行-验证智能体的增强状态数据结构
class PEVState(TypedDict):
    """
    PEV架构状态数据结构
```

```
    相比基础PE状态，新增了以下字段：
    - last_tool_result: 存储最近一次工具执行结果，供验证器检查
    - retries: 追踪重试次数，实现重试限制机制
    """
    user_request: str                       # 原始用户请求
    plan: Optional[List[str]]               # 待执行的计划步骤列表
    last_tool_result: Optional[str]         # 最近一次工具执行结果（用于验证）
    intermediate_steps: List[str]           # 已验证成功的中间结果列表
    final_answer: Optional[str]             # 最终合成的答案
    retries: int                            # 重规划计数器，防止无限重试

# 导入LangChain的解析异常，用于处理LLM输出格式错误
from langchain_core.exceptions import OutputParserException

# ==================== 增强计划模型 ====================
# 定义带有约束的计划模型，限制步骤数量防止无限扩展
class Plan(BaseModel):
    """
    增强计划模型
    新增约束：max_items=5，限制计划步骤最大数量，防止计划节点生成过于复杂的计划，控制执行时间
    """
    steps: List[str] = Field(description="List of queries (max 5).", max_items=5)

# ==================== PEV计划节点 ====================
def pev_planner_node(state: PEVState):
    """
    PEV架构计划节点 - 支持自适应重规划
    相比基础PE计划节点，新增功能：
    1. 重试限制：最多重规划3次，防止无限循环
    2. 错误恢复：能够基于失败历史重新制定计划
    3. 严格解析：使用strict=True强制输出格式

    参数:state: 当前PEV状态，包含执行历史和重试次数
    返回:dict: 包含新计划和递增重试次数的状态更新
    """
    # 获取当前重试次数，默认0
    retries = state.get("retries", 0)

    # 重试限制检查：最多3次重规划——防止系统陷入无限循环的关键保护机制
    if retries > 3:
        console.print("--- (PEV) PLANNER: Retry limit reached. Stopping. ---")
        return {
            "plan": [],                                         # 清空计划，终止执行
            "final_answer": "Error: Unable to complete task after multiple retries."  # 返回错误信息
        }

    console.print(f"--- (PEV) PLANNER: Creating/revising plan (retry {retries})... ---")
```

```
    # 配置结构化LLM，strict=True强制输出符合模式
    planner_llm = llm.with_structured_output(Plan, strict=True)

    # 构建包含历史执行结果的提示词
    # 关键设计：将之前的失败信息传递给计划器，避免重复相同错误
    past_context = "\n".join(state["intermediate_steps"])
    base_prompt = f"""
    You are a planning agent.
    Create a plan to answer: '{state['user_request']}'.
    Use the 'flaky_web_search' tool.

    Rules:
    - Return ONLY valid JSON in this exact format: {{ "steps": ["query1",
"query2"] }}
    - Maximum 5 steps.
    - Do NOT repeat failed queries or endless variations.
    - Do NOT output explanations, only JSON.

    Previous attempts and results:
    {past_context}
    """

    # JSON解析容错重试机制，尝试最多2次解析，失败时添加更严格的格式约束
    for attempt in range(2):
        try:
            plan = planner_llm.invoke(base_prompt)
            return {"plan": plan.steps, "retries": retries + 1}
        except OutputParserException as e:
            console.print(f"[red]Planner parsing failed (attempt
{attempt+1}): {e}[/red]")
            # 强化提示词，强制要求JSON格式
            base_prompt = f"Return ONLY valid JSON with {{'steps':
['...']}}. {base_prompt}"

    # 最终降级方案：返回一个默认计划，避免系统崩溃—保证系统鲁棒性的最后防线
    return {"plan": ["Apple R&D spend last fiscal year"], "retries":
retries + 1}

# ==================== PEV执行节点 ====================
def pev_executor_node(state: PEVState):
    """
    PEV架构执行节点 - 增强空计划保护
    相比基础PE执行节点，新增：
    - 空计划保护：当plan为空时跳过执行，避免索引错误
    - 单步执行：每次只执行一个步骤，交给验证器检查

    参数:state: 当前PEV状态，包含待执行计划
    返回:dict: 更新后的计划（移除已执行步骤）和工具执行结果
    """
    # 防御性编程：检查计划是否为空
    if not state.get("plan"):
        console.print("--- (PEV) EXECUTOR: No steps left, skipping
execution. ---")
        return {}
```

```
    console.print("--- (PEV) EXECUTOR: Running next step... ---")
    next_step = state["plan"][0]                    # 取出第一个步骤
    result = flaky_web_search(next_step)            # 执行工具调用

    # 返回状态更新:
    # - plan: 移除已执行步骤
    # - last_tool_result: 保存结果供验证器检查
    return {"plan": state["plan"][1:], "last_tool_result": result}

# ==================== 验证节点 ====================
def verifier_node(state: PEVState):
    """
    验证节点 - PEV架构的核心创新，负责检查工具执行结果的有效性，实现:
    1. 质量门控: 只有验证通过的步骤才被加入intermediate_steps
    2. 失败检测: 识别工具错误、数据不完整等问题
    3. 触发重规划: 失败时清空计划，触发重新规划

    参数:state: 当前PEV状态，包含最近一次工具执行结果
    返回:dict: 根据验证结果更新状态
        - 成功: 将结果加入intermediate_steps
        - 失败: 清空计划（触发重规划），记录失败信息
    """
    console.print("--- VERIFIER: Checking last tool result... ---")

    # 配置验证器LLM为结构化输出
    verifier_llm = llm.with_structured_output(VerificationResult)

    # 构建验证提示词
    prompt = f"Verify if the following tool output is a successful result
or an error message. The task was '{state['user_request']}'.\n\nTool
Output: '{state['last_tool_result']}'"

    # 执行验证
    verification = verifier_llm.invoke(prompt)

    console.print(f"--- VERIFIER: Judgment is '{'Success' if
verification.is_successful else 'Failure'}' ---")

    if verification.is_successful:
        # 验证成功: 将结果加入已验证的成功步骤列表
        return {"intermediate_steps": state["intermediate_steps"] +
[state['last_tool_result']]}
    else:
        # 验证失败: 清空计划（触发重规划），记录失败信息
        # 关键设计: 清空plan + 记录失败信息 → 路由节点将触发重规划
        return {
            "plan": [],
            "intermediate_steps": state["intermediate_steps"] +
[f"Verification Failed: {state['last_tool_result']}"]
        }

# ==================== 综合节点复用 ====================
# PEV架构复用基础PE的综合节点，保持接口一致性
```

```
pev_synthesizer_node = basic_synthesizer_node

# ==================== PEV路由决策节点 ====================
def pev_router(state: PEVState):
    """
    PEV架构路由决策节点 - 实现自适应控制流，根据当前状态决定下一步执行路径：
    1. 最终答案已存在 → 综合节点（终止）
    2. 计划为空且因验证失败导致 → 计划节点（重规划）
    3. 计划为空且正常完成 → 综合节点（输出结果）
    4. 计划非空 → 执行节点（继续）
    实现了PEV架构的核心循环：计划 → 执行 → 验证 → {成功:继续/失败:重规划} → 综合

    参数:state: 当前PEV状态
    返回:str: 下一个节点名称（"plan" / "execute" / "synthesize"）
    """
    # 优先检查：如果已有最终答案，直接进入综合阶段
    if state.get("final_answer"):
        console.print("--- ROUTER: Final answer available. Moving to
synthesizer. ---")
        return "synthesize"

    # 检查计划是否为空
    if not state["plan"]:
        # 判断计划为空的原因：是否为验证失败导致的清空
        # 通过检查最近一条记录是否包含"Verification Failed"判断
        if state["intermediate_steps"] and "Verification Failed" in
state["intermediate_steps"][-1]:
            console.print("--- ROUTER: Verification failed. Re-planning...
---")
            return "plan"          # 验证失败：回到计划节点重新规划
        else:
            console.print("--- ROUTER: Plan complete. Moving to
synthesizer. ---")
            return "synthesize"    # 计划正常完成：进入综合节点
    else:
        console.print("--- ROUTER: Plan has more steps. Continuing
execution. ---")
        return "execute"           # 还有剩余步骤：继续执行

# ==================== PEV图结构构建 ====================
# 构建增强的计划-执行-验证计算图
pev_graph_builder = StateGraph(PEVState)

# 添加四个核心节点
pev_graph_builder.add_node("plan", pev_planner_node)            # 计划节点
（支持重规划）
pev_graph_builder.add_node("execute", pev_executor_node)        # 执行节点
（单步执行）
pev_graph_builder.add_node("verify", verifier_node)             # 验证节点
（质量门控）
pev_graph_builder.add_node("synthesize", pev_synthesizer_node)  # 综合节点
（答案生成）

# 设置入口点：从计划节点开始
```

```
pev_graph_builder.set_entry_point("plan")
# 添加固定边：计划 → 执行 —— 计划节点执行后总是进入执行阶段
pev_graph_builder.add_edge("plan", "execute")
# 添加固定边：执行 → 验证 —— 执行节点执行后总是进入验证阶段
pev_graph_builder.add_edge("execute", "verify")
# 添加条件边：验证 → 路由决策 —— 验证结果决定下一步：重规划 / 继续执行 / 综合
pev_graph_builder.add_conditional_edges("verify", pev_router)
# 添加固定边：综合 → 终止
pev_graph_builder.add_edge("synthesize", END)

# ==================== 图编译与应用初始化 ====================
# 将构建的PEV图结构编译为可执行的应用实例
pev_agent_app = pev_graph_builder.compile()
# 输出编译成功状态
print("Planner-Executor-Verifier (PEV) agent compiled successfully.")
```

阶段 3：直接对比

现在进行关键性测试。在**相同的“不稳定”任务**上运行鲁棒的 PEV 智能体，观察它如何成功应对工具故障。

```
# ==================== PEV架构测试执行 ====================
# 对PEV（计划-执行-验证）智能体进行相同不稳定查询的测试，旨在验证PEV架构相比基础PE架构
在处理工具故障时的改进效果

# 输出测试开始标识，使用绿色区分基础PE测试的黄色标识
console.print(
    f"[bold green]Testing PEV agent on the same flaky query:[/bold
green]\n'{flaky_query}'\n"
)

# ==================== 初始化PEV状态 ====================
# 构建PEV智能体的初始状态，与基础PE状态相比，新增了retries字段用于追踪重规划次数
initial_pev_input = {
    "user_request": flaky_query,        # 原始用户请求（与基础PE测试相同）
    "intermediate_steps": [],           # 已验证成功的中间结果列表（初始为空）
    "retries": 0                        # 重规划计数器初始化为0，开始第一次规划
}

# ==================== PEV系统调用 ====================
# 执行PEV图应用，处理包含不稳定因素的复杂查询，触发PEV架构的自适应执行流程
#
# 执行流程详解
# 【第一次尝试】
# 阶段1 - 计划（PLANNER - retry=0）
#   分解任务
# 阶段2 - 执行（EXECUTOR）
#   执行步骤1: 查询研发支出 → 成功，返回财务数据
#   保存结果到 last_tool_result
# 阶段3 - 验证（VERIFIER）
#   检查结果: 是否为有效数据?
```

```
#   判断: 成功 → 将结果加入 intermediate_steps
# 阶段4 - 路由 (ROUTER)
#   plan还有步骤 → 继续执行
# 阶段2 - 执行 (EXECUTOR) - 第二次执行
#   执行步骤2: 查询员工总数 → 触发故障, 返回错误信息
#   保存错误结果到 last_tool_result
# 阶段3 - 验证 (VERIFIER)
#   检查结果: 是否为有效数据?
#   检测到错误信息 (API failure)
#   判断: 失败 → 清空plan, 记录失败信息
# 阶段4 - 路由 (ROUTER)
#   plan为空 + 检测到"Verification Failed"
#   决策: 回到PLANNER重新规划
#
# 【第二次尝试 (重规划)】
# 阶段1 - 计划 (PLANNER - retry=1)
#   接收历史: "Verification Failed: employee count"
#   智能调整: 避免重复失败的查询
#   新计划: 可能的替代策略 (如搜索替代数据源)
#
# ... 继续执行直到成功或达到重试限制 ...
#
# 【最终阶段】
# 综合阶段 (SYNTHESIZER)
#   收集所有已验证成功的中间结果, 生成最终答案
final_pev_output = pev_agent_app.invoke(initial_pev_input)

# ==================== 结果输出与渲染 ====================
# 输出最终答案的分隔标识, 使用绿色与基础PE的红色形成视觉对比
console.print("\n--- [bold green]Final Output from PEV Agent[/bold green] -
--")

# 使用Markdown渲染PEV智能体生成的最终答案
# 期望的输出行为:
# - 如果重规划成功获取员工数据: 显示完整的研发支出/员工计算
# - 如果员工数据仍然无法获取: 可能显示研发支出数据 + 说明员工数据获取失败
# - 如果达到重试限制: 返回错误信息
console.print(Markdown(final_pev_output['final_answer']))
```

> **输出结果讨论：** 成功！执行轨迹讲述了一个具备韧性的故事：
>
> 1. **首次规划：** 智能体初始创建的计划与基础智能体的类似。
> 2. **执行与失败：** 它成功执行了第一步，但在第二步（员工数量）上失败，**收到错误信息**。
> 3. **校验与捕获：** **校验器节点接收到错误信息**，其大语言模型正确判断这是一个失败的步骤（`is_successful: False`）。它将此失败信息添加到状态中。
> 4. **路由与重新规划：** **路由器**看到校验失败后，将执行流程返回至**规划器**。
> 5. **二次规划：** **规划器**现在知晓了之前的失败情况，创建了一个**新的、更智能的计划**。它可能会**尝试不同的搜索查询**，例如“苹果公司全球员工数量”，以规避 API 故障。
> 6. **执行与成功：** 它执行新计划，这次成功了。
> 7. **校验与通过：** **校验器确认新数据有效**。
> 8. **合成：** **合成器只接收到有效数据**，并生成了正确的最终答案。

> 这清晰地展示了 PEV 架构的**自我修正循环**如何使其能够克服那些会让更简单的智能体完全崩溃的障碍。

阶段 4：定量评估

最后使用“**大语言模型即评审**”的方式，对两种智能体在鲁棒性和错误处理能力方面进行评分。

```
# ==================== 鲁棒性评估模型定义 ====================
# 定义智能体鲁棒性和错误处理能力的评估模型——专门用于评估智能体在遇到工具故障时的表现
class RobustnessEvaluation(BaseModel):
    """
    智能体鲁棒性评估模型
    该评估框架从两个维度评估智能体的容错能力：
    1. 任务完成度：评估是否成功完成任务（忽略数据错误的影响）
    2. 错误处理能力：评估检测错误并从中恢复的能力

    属性:
    1. task_completion_score: 任务完成度评分（1-10）
       评估智能体是否在工具故障情况下仍能推进任务，即使最终答案不完美，只要尝试了合理处理
就给予较高分数
    2. error_handling_score: 错误处理能力评分（1-10）
       评估智能体是否：正确识别工具错误、采取适当的恢复措施（重规划、替代方案等）、向用户
清晰传达错误信息
    3. justification: 评分的详细理由
       提供评估决策的可解释性，帮助理解评分依据
    """
    task_completion_score: int = Field(
        description="Score 1-10 on whether the agent successfully completed
the task, ignoring data errors."
    )
    error_handling_score: int = Field(
        description="Score 1-10 on the agent's ability to detect and
recover from errors."
    )
    justification: str = Field(
        description="A brief justification for the scores."
    )

# ==================== 评估LLM配置 ====================
# 将LLM配置为结构化输出模式，强制返回RobustnessEvaluation格式，确保了评估结果的一致
性，便于后续分析和比较
judge_llm = llm.with_structured_output(RobustnessEvaluation)

# ==================== 智能体鲁棒性评估函数 ====================
def evaluate_agent_robustness(query: str, final_state: dict) ->
RobustnessEvaluation:
    """
    智能体鲁棒性评估函数 - 量化评估智能体的容错能力
    评估方法论：提取智能体的完整执行轨迹 -> 检查智能体是否检测到错误 -> 评估错误恢复机
制的有效性 -> 综合评估整体表现

    参数:query: 原始用户请求，作为评估的参考基准
        final_state: 智能体的最终状态，包含：
```

```
            - intermediate_steps: 执行过程中的所有步骤和结果
            - final_answer: 最终输出的答案
    返回:RobustnessEvaluation: 包含任务完成度和错误处理能力的评估结果
    """

    # 提取执行轨迹和最终答案
    # intermediate_steps 记录了智能体执行的每一步，包括成功和失败的结果
    context = "\n".join(final_state.get("intermediate_steps", []))
    final_answer = final_state.get("final_answer", "")

    # 构建完整的执行轨迹描述,便于评估器理解智能体的完整行为
    trace = f"Context:\n{context}\n\nFinal Answer:\n{final_answer}"

    # 构建评估提示词
    # 明确告知评估器：工具被设计为对特定查询失败,确保评估器理解测试场景的特殊性
    prompt = f"""You are an expert judge of AI agents. A tool used by the
agent was designed to fail on a specific query. Evaluate the agent's
ability to handle this failure.

    **User's Task:** {query}
    **Full Agent Trace:**\n```\n{trace}\n```
    """

    # 调用结构化LLM进行评估
    return judge_llm.invoke(prompt)

# ==================== 基础PE智能体鲁棒性评估 ====================
# 对基础计划-执行架构的智能体进行鲁棒性评估
# 该评估提供了基准数据，用于对比PEV架构的改进效果
console.print("--- Evaluating Basic P-E Agent's Robustness ---")

# 调用评估函数，传入原始查询和基础PE智能体的最终状态
# final_pe_output 包含：
#   - intermediate_steps: 包含工具执行结果（包括错误信息）
#   - final_answer: 基于所有结果（包括错误）生成的答案
pe_agent_evaluation = evaluate_agent_robustness(flaky_query,
final_pe_output)

# 输出评估结果
console.print(pe_agent_evaluation.model_dump())

# ==================== PEV智能体鲁棒性评估 ====================
# 对计划-执行-验证架构的智能体进行鲁棒性评估
# 与基础PE使用相同的评估标准，确保公平比较
console.print("\n--- Evaluating PEV Agent's Robustness ---")

# 调用评估函数，传入原始查询和PEV智能体的最终状态
# final_pev_output 包含：
#   - intermediate_steps: 只包含验证通过的成功结果
#   - final_answer: 仅基于验证成功的数据生成
pev_agent_evaluation = evaluate_agent_robustness(flaky_query,
final_pev_output)
```

```
# 输出评估结果
console.print(pev_agent_evaluation.model_dump())
```

> **输出结果讨论：** 评审模型的评分呈现出鲜明对比。 **基础规划器-执行器智能体**将获得**非常低的错误处理得分**，因为它**未能识别工具错误**，并生成了毫无意义的最终答案。相比之下，**PEV智能体**将获得**接近满分的错误处理得分**。评审模型的评语将肯定其**检测失败、触发重新规划循环，并最终恢复以提供正确答案**的能力。
>
> 这一评估定量地证明了 PEV 架构的价值。其意义不仅在于在一切顺利时能得到正确答案，更在于**在出现问题时不会给出错误答案**。

结论

在本节笔记本中，我们实现了**规划器 → 执行器 → 校验器**架构，并展示了其相较于简单**规划器-执行器模型**的显著鲁棒性优势。通过引入专门的**校验器节点**，我们为智能体配备了一个关键的“免疫系统”，使其能够检测并从中断任务执行的故障中恢复。

这种模式**在资源消耗上更高**，但对于那些**可靠性和准确性**至关重要的应用而言，这种权衡是必要的。PEV架构标志着我们在构建真正可信赖的AI智能体方面迈出了重要一步，使它们能够在**外部工具和API**构成的不可预测的现实环境中**安全、有效地运行**。